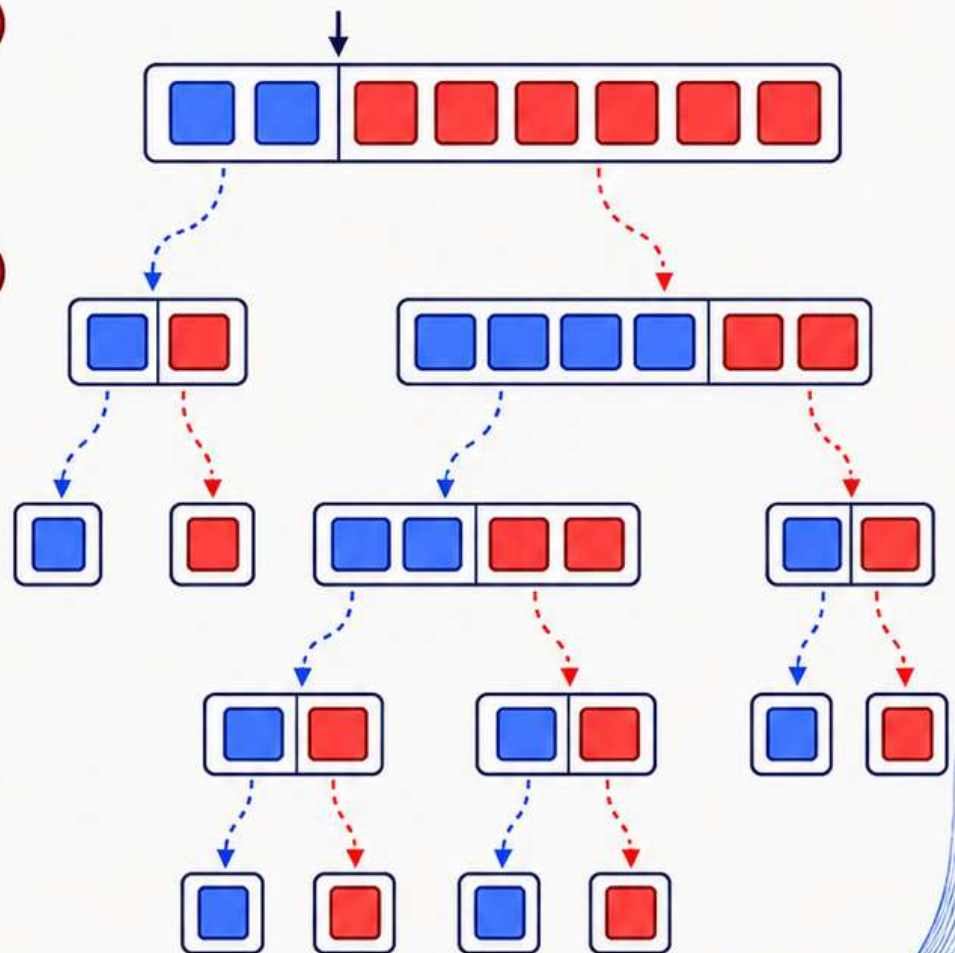
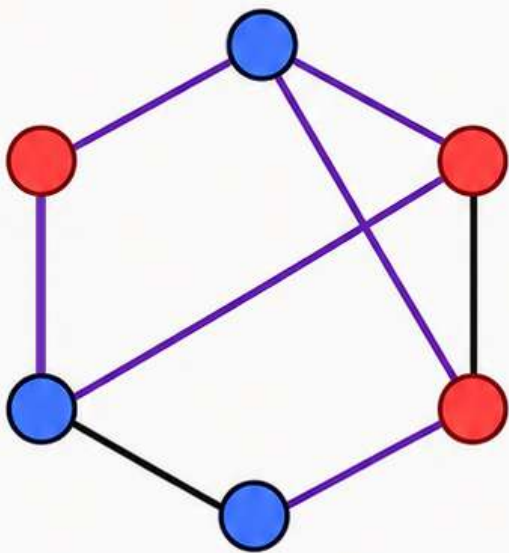


Randomised Algorithms

Sahil M Deo



Contents

1	Introduction	6
1.1	What is a Randomised Algorithm?	6
1.2	Implementation of Randomness in C++	6
1.3	Problems	9
2	Algorithmic Performance	10
2.1	Operations	10
2.2	Worst Case Analysis	10
2.3	Average Case Analysis	11
2.4	Time Complexity Analysis	11
2.5	Problems	13
3	Complexity Classes	14
3.1	Decision Problems	14
3.2	P	14
3.3	NP	15
3.4	Polynomial-Time Reductions	15
3.5	NP-Complete	16
3.6	NP-Hard	16
3.7	RP	17
3.8	ZPP	17
3.9	BPP	18
3.10	Summary	18
3.11	Problems	19
4	The Classics	19
4.1	QuickSort	19
4.1.1	The Algorithm	19
4.1.2	Worst Case Runtime	19
4.1.3	Expected Runtime	20
4.2	QuickSelect	21
4.2.1	The Algorithm	21
4.2.2	Expected Runtime	22
4.2.3	Expected Recursion Depth	22
4.3	Problems	24
5	Hashing	25
5.1	What even is Hashing?	25
5.2	Polynomial Multiplication	25
5.3	Matrix Multiplication	26
5.3.1	Freivalds' Algorithm	26

5.4	A Matrix Problem	27
5.4.1	Set of Sets	28
5.4.2	Hashing	29
5.5	A Sliding Window Problem	30
5.6	Problems	32
6	Data Structures	33
6.1	Skip Lists	33
6.1.1	The Structure	35
6.1.2	Expected Height	35
6.1.3	Search	36
6.1.4	Insertion	38
6.2	Treaps	39
6.2.1	Binary Trees	39
6.2.2	Tree Traversal	40
6.2.3	Binary Search Trees	40
6.2.4	Balancing	44
6.2.5	Min-Heaps	45
6.2.6	Rotations	48
6.2.7	Treaps	49
6.3	Hash Tables	53
6.3.1	The Structure	53
6.3.2	Insertion	53
6.3.3	Search	54
6.3.4	Deletion	55
6.4	Bloom Filters	55
6.4.1	The Structure	55
6.4.2	Insertion	55
6.4.3	Search	56
6.4.4	Utility of Bloom Filters	56
6.5	Cuckoo Hashing	56
6.5.1	The Structure	56
6.5.2	Insertion	57
6.5.3	Search	57
6.6	Problems	58
7	Number-Theoretic Algorithms	59
7.1	Introduction	59
7.2	Fermat Test	59
7.2.1	Fermat's Little Theorem	60
7.2.2	The Algorithm	60
7.2.3	Error Probability	60
7.3	Miller–Rabin Test	61
7.3.1	A Key Property	61
7.3.2	The Algorithm	62
7.3.3	Error Probability	62
7.3.4	Performance	63
7.4	Pollard's Rho Algorithm	63
7.4.1	Floyd's Cycle-Detection Algorithm	64
7.4.2	The Algorithm	66
7.4.3	Performance	67
7.5	Problems	70

8	Probabilistic Method	71
8.1	Introduction	71
8.2	Array Construction	71
8.2.1	Random Experiment	71
8.2.2	Success Probability	72
8.2.3	The Algorithm	73
8.2.4	Expected Time Complexity	74
8.3	Dominating Set	74
8.3.1	Random Experiment	75
8.3.2	Success Probability	75
8.3.3	The Algorithm	76
8.3.4	Expected Time Complexity	76
8.4	Bipartite Subgraph	77
8.4.1	Random Experiment	77
8.4.2	The Algorithm	78
8.4.3	Success Probability	78
8.4.4	Expected Time Complexity	80
8.4.5	Conjecture	80
8.4.6	Derandomisation	80
9	Online Algorithms	82
9.1	Introduction	82
9.2	Dating	82
9.3	Load Balancing	85
9.4	Distinct Elements	87
9.4.1	Flajolet–Martin Method	87
9.4.2	HyperLogLog Method	89
10	Heuristic Algorithms	90
10.1	Introduction	90
10.2	Dominating Set Revisited	90
10.3	Independent Set	92
10.3.1	Random Permutation	92
10.3.2	Local Decision	93
10.4	Bipartite Subgraph Revisited	95
10.4.1	Self-Correction	95
11	Game Theoretic Algorithms	96
11.1	Definitions	96
11.2	Nash Equilibria	96
11.2.1	Kingdom’s Dilemma	97
11.2.2	Penalty Kick	97
11.3	Mixed Strategies	98
11.4	Mixed Strategy Nash Equilibria	98
11.5	Indifference Theorem	99
11.6	The Minimax Principle	99
11.7	Problems	100

12 Zero-Knowledge Proofs	101
12.1 Definitions	101
12.2 Card Color	102
12.2.1 The Scheme	102
12.2.2 Completeness and Soundness	102
12.3 Graph Isomorphism	102
12.3.1 The Scheme	103
12.3.2 Soundness and Completeness	103
12.3.3 Game Theory Perspective	104
Appendices	105
A Probability and Expectation	106
A.1 Linearity of Expectation	106
A.2 Tail Sum Formula	106
A.3 Union Bound	106
A.4 Independence of Expectation	106
A.5 Variance	106
A.6 Indicator Variables	107
A.7 Law of Total Expectation	107
A.8 Wald's Equation	107
B Inequalities	109
B.1 Exponential Inequality	109
B.2 Jensen's Inequality	109
B.3 Markov's Inequality	110
B.4 Bernoulli's Inequality	111
C Convergence and Limit Theorems	112
C.1 Identical and Independently Distributed Random Variables	112
C.2 Almost Surely	112
C.3 Almost Never	112
C.4 $\lim_{n \rightarrow \infty} X_n = Y$	113
C.5 Convergence in Probability	113
C.6 The Weak Law of Large Numbers	113
C.7 Almost Sure Convergence	114
C.8 A_n infinitely often	114
C.9 The Borel–Cantelli Lemmas	114
C.10 The Strong Law of Large Numbers	115
C.11 Convergence in Distribution	115
C.12 The Central Limit Theorem	116
D Discrete Mathematics	117
D.1 Pigeonhole Principle	117
D.2 Relations	117
D.2.1 Partial Orders	117
D.2.2 Equivalence Relations	118
D.3 Graph Theory	118
D.3.1 Degree	118
D.3.2 Bipartite Graph	118
D.3.3 Subgraph	118
D.3.4 Clique	118

E	Linear Algebra	119
E.1	Vector Spaces	119
E.2	Subspaces	119
E.3	Dimension of a Subspace	119
E.4	The Nullspace	120
E.5	Nullity and Rank	120
E.6	The Rank–Nullity Theorem	120

Chapter 3

Complexity Classes

What is the best time complexity for a problem? When can we hope to come up with a *fast* algorithm? This is exactly the point of defining complexity classes. While the classes themselves are defined rigorously, there are some very important¹ statements which have not yet been proved – they remain open problems! In practice, however, we have conjectures which we believe to be true for now...

3.1 Decision Problems

Complexity classes are traditionally defined using *decision problems*. A decision problem is a computational problem whose answer is either YES or NO.

- Given a graph G , is G connected?
- Given an integer m , is m prime?
- Given a graph G and an integer k , does G contain a clique² of size at least k ?

Although not all computational tasks are naturally decision problems, most can be reformulated as one. For example, the largest clique in a graph can be reformulated³ as "is there a clique of size at least k in G ?".

3.2 P

The class **P** consists of all decision problems that can be solved by a deterministic algorithm in at most polynomial time. Let Q be a decision problem and $T(Q)$ be the best time complexity⁴ of a deterministic algorithm for Q . Then $T(Q) \leq \Theta(n^c)$ for some constant $c \geq 1$ implies $Q \in \mathbf{P}$.

$$\mathbf{P} = \bigcup_{c \geq 1} \{Q : T(Q) \leq \Theta(n^c)\}$$

Decision Problem(Q)	Algorithm	Input	Input Size	Runtime (T(Q))
Primality Test	AKS	Integer	$m = \Theta(\log_2 n)$	$\Theta(m^{12})$
Graph Connectivity	BFS	Adjacency Matrix	$n = V \times V $	$\Theta(n)$

¹Such as the classic question of $\mathbf{P} \neq \mathbf{NP}$...

²Refer Appendix D.3.4

³Some of these reformulations are called *polynomial time reductions* which will be properly discussed later.

⁴This time complexity is in terms of the input length and not the input number itself – a number n when taken as input is represented by a bit string of size $m = \Theta(\log n)$. So an algorithm which takes $O(n)$ time will be considered as an exponential time algorithm since $\Theta(n) = \Theta(2^m)$. This is why the normal method of trying all numbers from 1 to $\sqrt{n}$ will not prove that primality testing of a number is in P .

3.3 NP

NP stands for non-deterministic polynomial time. What that really means is, if you perform the non-deterministic procedure called *guessing*, you can check your answer in polynomial time. **NP** does **not** stand for non-polynomial time. Formally, the class **NP** consists of all decision problems whose answer when **YES** can be verified in polynomial time. The algorithm which solved the problem can output a *certificate* along with **YES** – this certificate is information which can be used to verify in polynomial time that the answer is indeed **YES**.

Decision Problem (Q)	Certificate	Verification Time
Clique of size k	Set of k vertices	$\Theta(k^2)$
Graph Coloring of k colors	Vertex : Color mapping	$\Theta(V + E)$

An obvious conclusion from the definition of **NP** is that $\mathbf{P} \subseteq \mathbf{NP}$ because any problem that can be solved in polynomial time can also be verified in polynomial time by giving the problem itself as a certificate and the verification procedure is the solution itself.

NP_{co}

The class **NP_{co}** contains complements of problems in **NP**. In this class of problems, the answer **NO** has a certificate verifiable in polynomial time. For example, the complement of the clique problem asks whether every clique in the graph has size at most $k - 1$. The answer **NO** can be accompanied by a certificate which is the set of vertices forming a clique of size at least k .

An obvious conclusion is that any algorithm that solves an **NP** problem also immediately solves the corresponding **NP_{co}** problem by just flipping the answer from **YES** to **NO** and vice versa but the catch being it may not be able to produce a certificate for **NO** answer.

3.4 Polynomial-Time Reductions

To compare the difficulty of computational problems, we use the idea of *polynomial-time reductions*. A problem A is polynomial-time reducible to a problem B if we can convert A to B in polynomial time.

This is denoted by $A \leq_p B$. This means that if we can solve B in polynomial time, then we can also solve A in polynomial time. The notation suggests B is harder than A and one might find this confusing because we "reduced" A to B so shouldn't B be easier? Think of a reduction as a one-way procedure that converts A to a *particular* case of B . Solving the general case of a problem is at least as hard as solving the particular case. That is why B is at least as hard as A under polynomial time reduction.

The operator $\leq_p$ is a partial order⁵. We can also define $=_p$ as $A =_p B$ iff $A \leq_p B$ and $B \leq_p A$. This can be used to induce equivalence classes⁶ within the complexity classes like **NP – complete** which is the "biggest" equivalence class in **NP**.

⁵Refer Appendix D.2.1

⁶Refer Appendix D.2.2

3.5 NP-Complete

NP – complete consists of those **NP** problems which are at least as hard as *every* problem in **NP**. For an L in **NP – complete**, every problem in **NP** can be polynomially reduced to L .

$$\mathbf{NP - complete} = \{L \in \mathbf{NP} : Q \leq_p L \quad \forall Q \in \mathbf{NP}\}$$

3.6 NP-Hard

NP – hard consists of all those problems which are harder than *every* problem in **NP**. These problems need not even have polynomially verifiable certificates for **YES**. The problems need not even be decision problems – they could be any type of problem like optimisation or search problems.

$$\mathbf{NP - hard} = \{L : L \leq_p Q \quad \forall Q \in \mathbf{NP}\}$$

An obvious conclusion is that $\mathbf{NP - complete} = \mathbf{NP} \cap \mathbf{NP - hard} \subseteq \mathbf{NP - hard}$.

Example

The κ -SAT decision problem is defined as follows – given a Boolean formula ϕ in the form $(k - \text{CLAUSE}_1) \text{ AND } (k - \text{CLAUSE}_2) \text{ AND } \dots$ (where a $k - \text{CLAUSE}$ is defined as k boolean variables joined using OR), determine whether there exists a satisfying assignment, that is, whether each variable in ϕ can be assigned a value to make $\phi = 1$. An example of a κ -SAT expression with $k = 2$ is $\phi = (x \vee \neg z) \wedge (\neg x \vee y) \wedge (y \vee w)$

For $k = 2$, the problem 2-SAT can be polynomially reduced to the problem of SCC (Strongly Connected Graphs) by considering each variable as a node and each clause as inducing edges. SCC has been solved in polynomial time using DFS based approach. Now, given an instance of 2-SAT, replace each clause $a \vee b$ by $(a \vee b \vee z) \wedge (a \vee b \vee \neg z)$ where z is a fresh variable that does not appear elsewhere in the formula. These two expressions are logically equivalent, so this is a valid reduction. This reduction can clearly be performed in polynomial time.

$$\implies 2\text{-SAT} \leq_p \text{SCC} \quad \text{and} \quad 2\text{-SAT} \leq_p 3\text{-SAT}$$

In fact, from any clause $(a_1 \vee a_2 \vee \dots \vee a_n)$ we can go to $(a_1 \vee a_2 \vee \dots \vee a_n \vee z) \wedge (\neg z \vee a_1 \vee a_2 \vee \dots \vee a_n)$

$$\implies 2\text{-SAT} \leq_p 3\text{-SAT} \leq_p \dots \iff 2\text{-SAT} \leq_p 3\text{-SAT} \leq_p \kappa\text{-SAT} \quad \forall k$$

All the $\kappa\text{-SAT} \in \mathbf{NP}$ because the satisfying assignment is checkable in polynomial time. Of course, $2\text{-SAT} \in \mathbf{P}$ as well as **NP** because of the reduction to SCC.

What about the bigger κ -SATs? For a general n -SAT, $(a_1 \vee a_2 \vee \dots \vee a_n)$ can be rewritten as $(a_1 \vee a_2 \vee b_1) \wedge (\neg b_1 \vee a_3 \vee b_2) \wedge (\neg b_2 \vee a_3 \vee b_3) \wedge \dots \wedge (\neg b_{n-3} \vee a_{n-1} \vee a_n)$. This introduced $n - 3$ new variables $b_1, b_2, \dots, b_{n-3}$ and made it into a κ -SAT.

$$\kappa\text{-SAT} \leq_p 3\text{-SAT}$$

Since we proved both $\kappa\text{-SAT} \leq_p 3\text{-SAT}$ and $3\text{-SAT} \leq_p \kappa\text{-SAT}$, we have proved that $\kappa\text{-SAT} =_p 3\text{-SAT}$. With some more effort, it can be shown that all of **NP** is reducible to 3-SAT. Thus 3-SAT is **NP – complete** and with it, all κ -SAT.

$$\underbrace{\text{SCC} = 2\text{-SAT} \leq_p 3\text{-SAT}}_{\mathbf{P}} = \underbrace{4\text{-SAT} = 5\text{-SAT} = \dots}_{\mathbf{NP} - \text{complete}}$$

P and **NP – complete** are quite close in difficulty, or are they? It is conjectured (and widely believed) that they are not equal. For more on this topic, see end-of-chapter problems!

3.7 RP

A decision problem belongs to **RP** if there exists a randomised polynomial-time algorithm with input size n satisfying the following properties:

- If the correct answer is **NO**, the algorithm always outputs **NO**.
- If the correct answer is **YES**, the algorithm outputs **YES** with probability at least $\frac{1}{n^c}$, $c > 0$.

Consider an algorithm as mentioned above – it outputs **YES** in the case where correct answer is **YES** with probability $p \geq \frac{1}{n^c}$. Run it k times and answer **YES** if it occurs even once in the k runs otherwise answer **NO**. This new algorithm has the same properties as the original algorithm except p is replaced by $1 - (1 - p)^k$. So if an algorithm has $p \geq \frac{1}{n^c}$, then for $k = n^c$ the probability will be more than $1 - \frac{1}{e}$.

$$\left(1 - \frac{1}{n^c}\right)^{n^c} \leq \frac{1}{e} \implies 1 - \left(1 - \frac{1}{n^c}\right)^{n^c} \geq 1 - \frac{1}{e}$$

As long as k is polynomial in the input size, the overall algorithm is also polynomial time. The point of having a lower bound $\frac{1}{n^c}$ is that we should be able to achieve any desired correctness probability by repeating the algorithm a polynomial number of times – this may not be possible if our criterion is something like $\frac{1}{2^n}$. Once we get a lower bound of $1 - \frac{1}{e}$, we can do a constant number of repetitions (still keeping the algorithm overall polynomial) to achieve a probability which is as close to 1 as we desire.

RP_{co}

A problem belongs to **RP_{co}** if there exists a randomised polynomial-time algorithm satisfying the following properties:

- If the correct answer is **YES**, the algorithm always outputs **YES**.
- If the correct answer is **NO**, the algorithm outputs **NO** with probability at least $\frac{1}{n^c}$, $c > 0$.

To clarify, when we say randomised polynomial time algorithm, we mean an algorithm which makes random choices but whose time complexity is polynomial in the input size. The relevant time complexity here is the actual worst-case time complexity and **not** the expected time complexity.

3.8 ZPP

A decision problem belongs to **ZPP** if there exists a randomized algorithm that has the following properties:

- It outputs the correct answer with probability 1.
- Expected time complexity is polynomial in the input size.

3.9 BPP

A decision problem belongs to **BPP** if there exists a randomized polynomial-time algorithm whose probability of producing the correct answer is at least $\frac{1}{n^c}$, where $c > 0$. Interestingly, it is believed that randomness does not truly add computation power and thus all the classes **P**, **RP**, **ZPP**, **BPP** are the same. This is conjecture, just like **P** $\neq$ **NP**.

3.10 Summary

Class	Time Complexity	YES	NO
P	Polynomial	Always correct	
NP	Verification in Polynomial	Certificate	Need not have certificate
RP	Polynomial	$\Pr(\text{correct}) \geq \frac{1}{n^c}$	Always correct
ZPP	Expected polynomial	Always correct	
BPP	Polynomial	Overall $\Pr(\text{correct}) \geq \frac{1}{n^c}$	

3.11 Problems

- 1 Solving an **NP – complete** problem in polynomial time is considered extremely difficult (if not impossible). Let C be the event that a single **NP – complete** problem is solved in polynomial time.
 - (a) Show that $C \implies \mathbf{P} = \mathbf{NP} = \mathbf{NP} - \text{complete}$
 - (b) Show that $\mathbf{P} \subseteq (\mathbf{NP}_{\text{co}} \cap \mathbf{NP}) \subseteq \mathbf{NP}$ and thus show $C \implies \mathbf{NP} = \mathbf{NP}_{\text{co}}$
- 2 Modern cryptography relies on the existence of one-way functions. One-way functions are functions which are computable in polynomial time but not invertible in polynomial time. Consider the function $F : \{0, 1\}^n \mapsto \{0, 1\}^m$
 - (a) Show that if F is invertible then $m \geq n$.
 - (b) Using the above result, show that if $\mathbf{P} = \mathbf{NP}$ then F can not be a one-way function by reformulating the problem of finding the inverse as a decision problem. *Hint: Go bit-by-bit.*
- 3 Consider an algorithm **ALGO** which solves a decision problem $Q \in \mathbf{ZPP}$ with runtime T whose expected value is E such that $E \leq n^c$ where $c > 0$. Now consider the algorithm **TIMED_ALGO**:

```

TIMED_ALGO
{
    run ALGO with time limit 2*E
    if time limit exceeded
        return NO
    else
        return answer of ALGO
}

```

Let p be the probability that **TIMED_ALGO** gives the correct answer.

- (a) Show that $p \geq \Pr(T \leq 2E)$
- (b) Use the markov inequality to show that $\Pr(T > 2E) \leq \frac{1}{2}$
- (c) Use the answers to (a) and (b) to show that the problem is solved by **TIMED_ALGO** with probability at least $\frac{1}{2}$.
- (d) Using the result from (c), show that $Q \in \mathbf{BPP}$. Show that this proof works for a general $Q \in \mathbf{ZPP}$, thus proving that $\mathbf{ZPP} \subseteq \mathbf{BPP}$
- (e) Use a similar setup to show $\mathbf{ZPP} \subseteq \mathbf{RP} \cap \mathbf{RP}_{\text{co}}$
- (f) For a general problem Z in $\mathbf{RP} \cap \mathbf{RP}_{\text{co}}$, use its **RP** solution and its **RP_{co}** solution to create a **ZPP** solution, thus showing that $\mathbf{RP} \cap \mathbf{RP}_{\text{co}} \subseteq \mathbf{ZPP}$. Use this along with (e) to prove that $\mathbf{RP} \cap \mathbf{RP}_{\text{co}} = \mathbf{ZPP}$